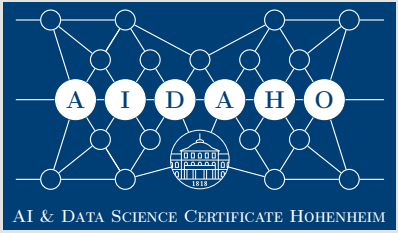
AIDAHO Tools Course: Tooling Up

A working, understandable, and inspectable Python/R environment

Dr. Markus Mößler

University of Hohenheim



Learning objectives

By the end of this session, you should be able to:

- access **bwJupyter** as our common fallback environment;
- run **Python and R scripts** on **bwJupyter** and/or **locally**; *("required setup")*
- work with **Jupyter Notebook/ RMarkdown/Quarto** documents; *("required setup")*
- distinguish **interpreter, packages, package manager, environment, IDE**;
- explain the difference between **local and remote execution**;
- use basic **Markdown and CLI** workflows;
- recognize why **diffs and version control** matter when humans and AI systems edit files.
- *The tooling-up for **SQL** will be treated in a separate session later in the course*

Success criterion

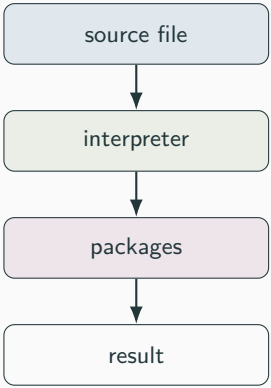
For a file such as `.py`, `.R`, `.ipynb`, or `.rmd`/`.qmd`, you can explain *what executes it, where packages come from, and where the computation runs*.

Why does code need an environment?

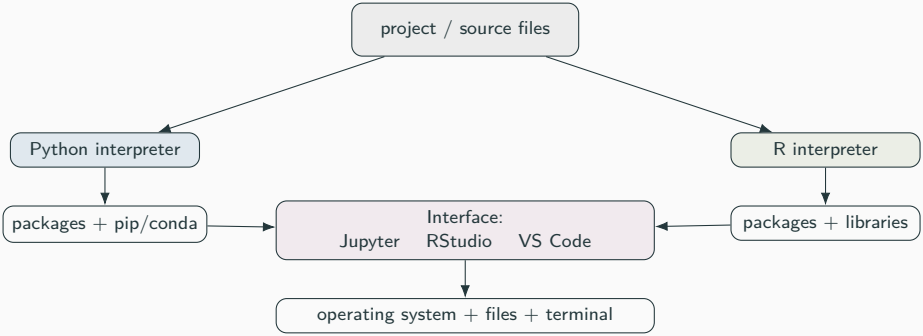
A source file is not the whole system

`analysis.py` or `report.R` is text. To turn it into results, we also need software that interprets the code and provides dependencies.

- Which interpreter runs the code?
- Which packages are available?
- Which version of those packages?
- Which interface launches the computation?



Anatomy of a coding environment



Key idea

Python, Anaconda, Jupyter, RStudio, VS Code, and a notebook are **different layers**.
Naming the layer makes troubleshooting much easier.

Local versus remote execution

	Remote: bwJupyter	**Local: your computer**
Setup	little/no local installation	installation and configuration required
Consistency	common course environment	may differ by OS
Role	reference/fallback environment	useful for own exploration/projects
Offline use	no	yes
Control	managed environment	high control
Best role	reliable common denominator	personal development environment

Course strategy

Remote guarantees participation; local builds independence.

"Best Case": Explore both routes available.

bwJupyter: our course reference environment

The Baden-Württemberg service provides a central web-based JupyterHub for teaching, including configurable profiles and support for distributing, collecting, and automatically grading learning materials (bwJupyter 2026a; bwJupyter 2026b).

- browser-based and device-independent;
- preconfigured environments reduce installation/version friction;
- shared course profiles create a common baseline;
- ideal location for automatically graded assignments.

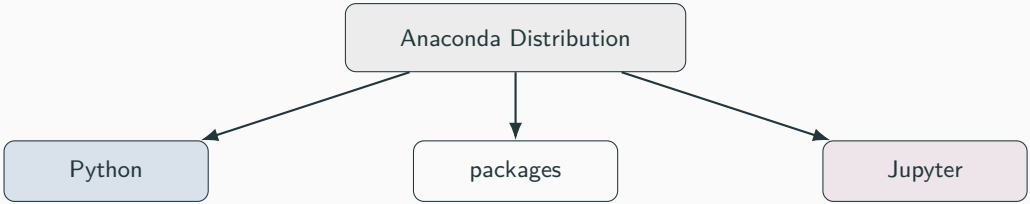
Not merely a fallback

Treat bwJupyter as the **reference environment**

remote Jupyter grading

A more detailed introduction to bwJupyter and Anaconda can be found in the Learning Module.

Python locally: what Anaconda gives us



Recommended local route

Anaconda Distribution bundles Python, conda, Jupyter Notebook/JupyterLab, and a large package ecosystem.

`analysis.py` → Python

What you need for the course

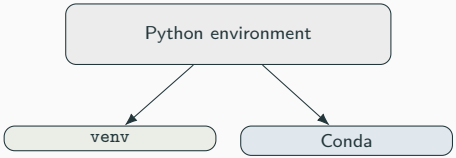
Install Anaconda, use the provided environment, and focus on **running Python code and notebooks**.

`analysis.ipynb` → Jupyter kernel

Zoom in: what is a Python environment?

Additional / under-the-hood knowledge

For this course, you need to understand the **idea** of an environment. You do **not** need to manage multiple environments yourself yet.



An environment is an **isolated combination** of a Python interpreter and installed packages.

Course perspective

Anaconda/Conda handles this layer for our recommended local setup. The important question is:

Which Python and packages are running my code?

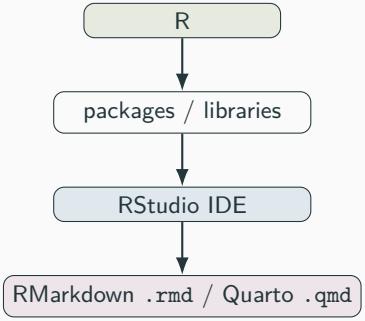
R: interpreter → packages → RStudio

Recommended local route

Install **R**, then **RStudio**. RStudio combines a console, source editor, plotting, debugging, and workspace tools (Posit Software, PBC 2026).

Computational documents

RMarkdown `.qmd` / Quarto `.qmd` files combine Markdown and executable code and can render to formats such as HTML, PDF, and Word (Quarto 2026).



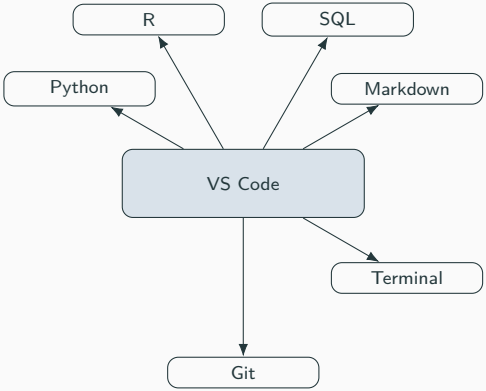
A more detailed introduction to R and RStudio can be found in the Learning Module.

VS Code: one interface for many tools

- Python, R, SQL, Markdown
- extension ecosystem
- integrated terminal
- Git/source-control integration
- notebook and RMarkdown/Quarto workflows

Positioning in this course

Optional / forward-looking. Jupyter and RStudio remain the primary beginner interfaces.



Markdown: documentation as text

```
# My first data project

## Goal
Investigate ...

## Files
- analysis.ipynb
- data.csv
- README.md
```

Why plain text?

- portable and tool-independent;
- easy to search and edit;
- easy to compare with a diff;
- works naturally with version control;
- inspectable when humans and AI systems collaborate.

The same Markdown ideas recur in Jupyter cells, RMarkdown/Quarto documents, and README files.

CLI: explicit, reproducible interaction

```
pwd
ls # Windows: dir
cd project
mkdir results
python analysis.py
Rscript analysis.R
code .
```

GUI vs. CLI

A GUI expresses actions through clicks.
A CLI expresses actions as **text**.

Why that matters

Commands are explicit, copyable, documentable, reproducible, and automatable.

AI literacy

LLMs and AI agents can propose commands. **Inspect and understand a command before executing it**, especially if it modifies or deletes files.

Computational files: what executes what?

File	Typical content	Execution	Typical interface
`.py`	Python code	Python interpreter	CLI / VS Code
`.ipynb`	cells + outputs + Markdown	Jupyter kernel	Jupyter / VS Code
`.R`	R code	R interpreter	CLI / RStudio / VS Code
`.rmd`/`.qmd`	cells + outputs + Markdown	RMarkdown / Quarto + engine	RStudio / VS Code
`.md`	plain-text documentation	usually not executed	VS Code

Question to ask

When something "does not run", is the problem in the **file**, the **interpreter/kernel**, a **package**, or the **interface**?

From `code --diff` to Git Version Control

VS Code's CLI can open a difference editor for two files (Microsoft 2026):

```
code --diff documentation-v1.md \
            documentation-v2.md
```



Four questions

Diff: What changed?
History: When/why?
Restore: Can we recover an earlier state?
Collaborate: Collaborate: How do we propose, review, and merge changes?

Git records project changes over time and supports comparison and recovery (Chacon and Straub 2026).

Next

Put the Git repository on GitHub/GitLab, and changes become things that humans – or AI-agents – can propose, review, discuss, and merge.

Tooling-up challenge (“Best-Case Scenario”)

For the “Best-Case Scenario”, demonstrate that you can:

1. log into bwJupyter and execute a notebook;
2. run a local Python script and a Jupyter notebook;
3. identify your Python environment and explain Conda vs. `venv`;
4. run an R script and render a small `.rmd`/`.qmd`;
5. create a `README.md`;
6. navigate to the project from a terminal and open it with `code .`;
7. compare two Markdown files with `code --diff`;
8. explain why diffs/version control matter when an AI system changes files.

Definition of “Best-Case Scenario”

You have a **working route** for the course and enough vocabulary to diagnose which layer is failing.

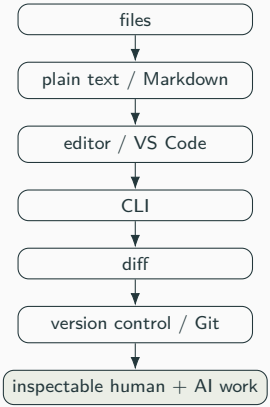
The strategy in one picture

Three operational layers

Understand	conceptual slides
Set up	detailed guides
Verify	tooling challenge

Desired outcome

Not tooling experts – students with a **working setup, clear mental model, and inspectable workflow**.



References i

bwJupyter (2026a). *bwJupyter for Education*. URL: https://www.bwjupyter.de/english/index.php (visited on 10/12/2026).

— (2026b). *Documentation*. URL: https://www.bwjupyter.de/english/65.php (visited on 10/12/2026).

Chacon, Scott and Ben Straub (2026). *Pro Git: About Version Control*. URL: https://git-scm.com/book/en/v2/Getting-Started-About-Version-Control.html (visited on 10/12/2026).

Microsoft (2026). *Visual Studio Code Command Line Interface*. URL: https://code.visualstudio.com/docs/configure/command-line (visited on 10/12/2026).

Posit Software, PBC (2026). *RStudio IDE*. URL: https://posit.co/downloads/ (visited on 10/12/2026).

Quarto (2026). *Tutorial: Hello, Quarto*. URL: https://quarto.org/docs/get-started/hello/rstudio.html (visited on 10/12/2026).